

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

К защите допустить:

Заведующий кафедрой ИИТ

_____ Д. В. Шункевич

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
по дисциплине: "Проектирование баз знаний"
на тему:

**РЕАЛИЗАЦИЯ ИНТЕРПРЕТАТОРА SC-МОДЕЛЕЙ
ПОЛЬЗОВАТЕЛЬСКИХ ИНТЕРФЕЙСОВ**

БГУИР КП5 1-40 03 01 01 059 ПЗ

Студент гр. 121703
Руководитель

И. С. Прокопович
М. Е. Садовский

Минск 2023

СОДЕРЖАНИЕ

Перечень условных обозначений	3
Введение	4
1 Анализ подходов к созданию веб-приложений	6
1.1 Анализ требований к разрабатываемой системе	6
1.2 Анализ аналогов разрабатываемой системы	7
1.3 Вывод	9
2 Проектирование разрабатываемой системы	11
2.1 Постановка задачи	11
2.2 Потенциальные пользователи разрабатываемой системы . . .	11
2.3 Архитектура разрабатываемой системы	12
2.4 Вывод	15
3 Разработка веб-интерфейса	16
3.1 Используемые средства разработки	16
3.2 Реализация функционала	17
3.3 Демонстрация реализованного функционала	21
3.4 Вывод	22
Заключение	24
Список использованных источников	26

ПЕРЕЧЕНЬ УСЛОВНЫХ ОБОЗНАЧЕНИЙ

В курсовом проекте используются следующие условные обозначения:

БЗ — база знаний;

OSTIS — Open Semantic Technologies for Intelligent Systems;

SC — Semantic Code;

SCg — Semantic Code graphical;

SCs — Semantic Code string;

SCn — Semantic Code natural.

ВВЕДЕНИЕ

В настоящее время информационные технологии играют важнейшую роль во многих сферах нашей жизни. Одной из таких сфер является область искусственного интеллекта, где разработка и применение интеллектуальных систем становятся все более распространенными. Интеллектуальные системы обладают возможностью анализировать данные, принимать решения и выполнять задачи с высокой точностью и эффективностью, демонстрируя уровень интеллектуальной активности, сравнимый с человеческим.

Однако для полноценного использования и взаимодействия с интеллектуальной системой необходим удобный и интуитивно понятный интерфейс. Веб-интерфейс является одним из наиболее популярных и доступных способов предоставления доступа к функциональности интеллектуальной системы. Он позволяет пользователям взаимодействовать с системой через веб-браузер, не требуя установки дополнительного программного обеспечения.

Тема данного курсового проекта – реализация интерпретатора SC-моделей пользовательских интерфейсов. Одной из подзадач является разработка и реализация веб-интерфейса для интеллектуальной системы. Веб-интерфейс должен предоставлять удобные и интуитивно понятные средства для взаимодействия с системой, позволяя пользователям использовать ее функциональность без необходимости в глубоких знаниях программирования или специализированного обучения. Главной задачей проекта является создание пользовательского интерфейса, который обеспечит эффективное и удобное взаимодействие пользователя с функциональными возможностями интеллектуальной системы.

Для достижения поставленной цели необходимо решить следующие задачи:

- изучить требования и функциональные возможности интеллектуальной системы;
- изучить основы разработки веб-интерфейсов, включая языки программирования, фреймворки и инструменты;
- разработать дизайн и структуру веб-интерфейса, учитывая требования пользователей и особенности интеллектуальной системы;
- реализовать веб-интерфейс, используя выбранные инструменты и технологии;
- протестировать и отладить разработанный веб-интерфейс, убедившись в его корректной работе и соответствии требованиям;
- предоставить документацию по использованию и сопровождению разработанного веб-интерфейса.

В результате успешной реализации данного проекта пользователи смогут взаимодействовать с интеллектуальной системой с помощью удобного

веб-интерфейса, получая доступ к ее функциональности и решая задачи с помощью интеллектуальных алгоритмов и анализа данных.

Кроме того, веб-интерфейс может предоставлять дополнительные возможности, такие как визуализация данных, отображение результатов анализа и обработки, настройка параметров системы и многое другое. Это позволяет пользователям более гибко управлять и взаимодействовать с интеллектуальной системой, расширяя ее функциональность и применение.

В заключение, разработка и реализация веб-интерфейса для интеллектуальной системы является важным шагом в обеспечении эффективного и удобного взаимодействия пользователей с системой. Успешная реализация такого интерфейса позволит раскрыть все возможности интеллектуальной системы и значительно улучшить пользовательский опыт, способствуя применению и принятию решений на основе анализа данных и интеллектуальных алгоритмов.

В настоящее время существует реализация системы, которая доступна по адресу ims.ostis.net. Однако, в рамках данного курсового проекта планируется разработать новую версию этой системы с использованием инновационных подходов и функциональности, чтобы улучшить ее производительность, надежность и пользовательский опыт.

Новая система должна предоставлять следующие возможности:

- выбор языка интерфейса SC-Web 0.9.0 (русский/английский);
- выбор языка представления выбранного фрагмента базы знаний (естественный язык, SCn, SCg);
- переход на страницу базы знаний IMS (главную страницу);
- поиск любой именованной сущности, имеющейся в базе знаний;
- выбор формата взаимодействия с базой знаний (мануальная навигация посредством нажатия на название соответствующей сущности, перемещение посредством задания вопросов системе);
- навигация по основным разделам базы знаний;
- свертывание списка разделов;
- навигация по истории запросов к базе знаний;
- свертывание истории запросов к базе знаний;
- навигация по базе знаний посредством задания вопросов системе;
- переход к семантической окружности любой сущности, имеющейся в базе знаний;
- навигация по базе знаний посредством вызова меню аргументов;
- закрепление вопросов к системе (аргументы) и задавание вопросов одновременно нескольким аргументам;
- изменение масштаба представляемого фрагмента базы знаний (при выборе SCg в качестве языка представления выбранного фрагмента);
- перемещение любых scg-элементы (узлы, дуги, контуры и др.).

На рисунке 1.1 проиллюстрировано описание требуемых возможностей пользователя в разрабатываемой системе.

1.2 Анализ аналогов разрабатываемой системы

В ходе работы был проведен анализ следующих аналогов:

1 Chatsonic.com [3] - это платформа чат-ботов с искусственным интеллектом, которая предоставляет простой и интуитивно понятный интерфейс для создания автоматических разговоров. Оно в основном ориентировано на конечных бизнес-пользователей, и с помощью этого приложения любой желающий может легко вести увлекательные беседы с клиентами всего за несколько минут. Компонент диалога изображен на рисунке 1.2.

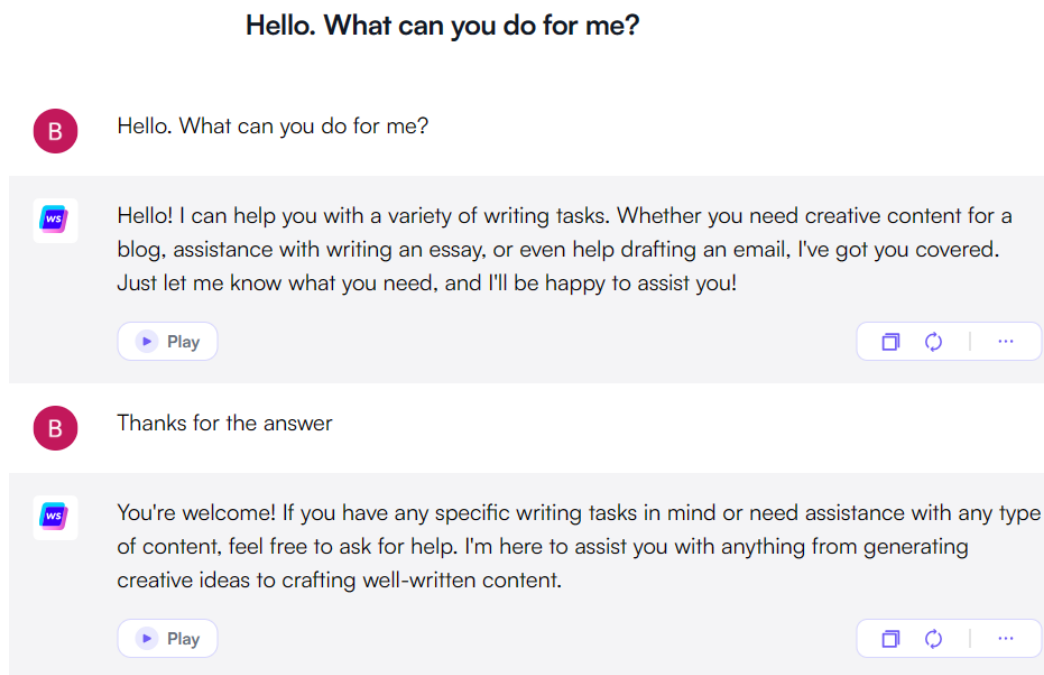


Рисунок 1.2 – Пример начального состояния системы "chatsonic.com"

2 Rytr.com [4] - это инструмент для генерации контента, который использует искусственный интеллект для создания текстов на основе предоставленных пользователем инструкций или примеров. Пользователи могут использовать Rytr для написания блогов, статей, рекламных текстов, электронных писем и других видов контента. Компонент диалога изображен на рисунке 1.3.

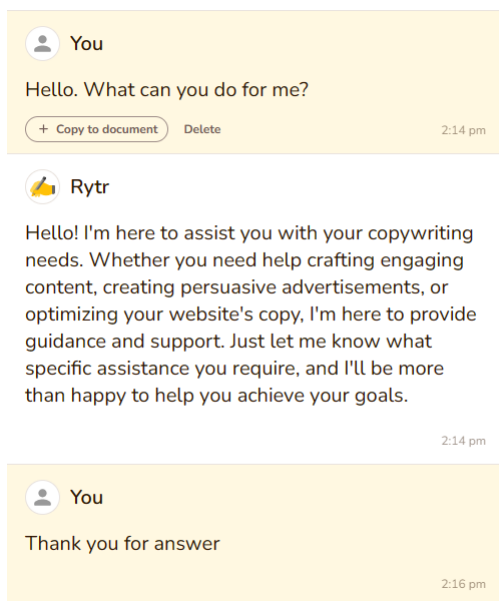


Рисунок 1.3 – Пример начального состояния системы "rytr.com"

Проанализировав аналоги можно выделить следующие характеристики и особенности диалоговых компонентов:

Chatsonic

- наличие функции задания вопросов с использованием микрофона;
- наличие функции прослушивания ответов;
- пользователи могут копировать ответы;
- пользователи могут запросить генерацию нового ответа.

Rytr

- наличие времени отправки запроса и ответа;
- пользователи могут копировать ответы или запросы;
- пользователи могут удалять ответы или запросы.

Все эти функции обеспечивают удобство использования и гибкость взаимодействия с системой.

В SC-Web компонент диалога не предоставляет такого обширного функционала как в рассматриваемых аналогах. Из наиболее важного функционала стоит отметить отсутствие запроса генерации нового ответа, что может ограничить возможности пользователя в получении более подходящей или улучшенной информации. Пользователям не предоставляется возможность копировать ответы для сохранения или дальнейшего использования. А также не предоставляется возможность задавать вопросы с использованием микрофона, что может усложнить процесс ввода информации, особенно в случае длительных переписок. В целом, отсутствие этих функций в системе может повлиять на удобство, гибкость и производительность взаимодействия с ней.

Следует отметить, что в SC-Web компонент диалога представлен в удобном и интуитивно понятном виде. Визуальный дизайн и расположение элементов обеспечивают комфортное чтение для пользователей, что способствует лучшему пониманию и повышению эффективности коммуникации.

1.3 Вывод

В результате проведенного анализа можно сделать следующие выводы:

1 Один из ключевых аспектов разработки - провести тщательное исследование аналогов, чтобы понять, какие функции и данные доступны в других языковых интерфейсах. Это позволит определить уникальные возможности и преимущества, которые можно внедрить в SC-Web. На данный момент в SC-Web компонент диалога не предоставляет такого обширного функционала как в рассматриваемых аналогах. Но на основе полученных данных, можно разработать дизайн и функциональность SC-Web, которые будут отвечать потребностям пользователей и предлагать уникальные возможности по сравнению с аналогами. Однако, SC-Web обладает некоторыми преимуществами, например интуитивный и понятный интерфейс, удобство для длительных переписок и легкость в чтении сообщений.

2 Разработка новой версии SC-Web позволит создать современное, эффективное и масштабируемое веб-приложение с улучшенным пользовательским опытом.

3 Были перечислены требования к разрабатываемой системе, которые позволяют пользователям качественно взаимодействовать с базой знаний.

2 ПРОЕКТИРОВАНИЕ РАЗРАБАТЫВАЕМОЙ СИСТЕМЫ

2.1 Постановка задачи

Разработка новой версии SC-Web должна включать следующие подзадачи:

1 Обновление архитектуры. Проанализировать текущую архитектуру SC-Web и определить области для улучшения и модернизации. Обновить архитектуру, чтобы обеспечить более эффективную обработку данных, масштабируемость и гибкость системы.

2 Расширение функциональности. Исследовать и внедрить новые возможности и функции, которые могут улучшить пользовательский опыт и расширить область применения SC-Web. Примеры включают поддержку дополнительных языков и форматов отображения данных.

3 Улучшение интерфейса пользователя. Продумать и реализовать новый интерфейс пользователя, который будет интуитивно понятным, легким в использовании и эстетически привлекательным. Уделить внимание удобству навигации, визуализации данных и общей пользовательской доступности.

4 Улучшение производительности. Оптимизировать производительность SC-Web, уменьшить время ответа на запросы пользователей и обеспечить более эффективное использование ресурсов системы. Это может включать оптимизацию алгоритмов, кэширование данных и использование параллельных вычислений.

5 Тестирование. Провести обширное тестирование новой версии SC-Web, чтобы обнаружить и исправить возможные ошибки, сбои и неполадки. Убедиться в надежности и стабильности системы перед её выпуском.

2.2 Потенциальные пользователи разрабатываемой системы

Потенциальные пользователи новой версии SC-Web могут включать в себя:

1 Исследователей и ученых из различных областей, таких как биология, физика, социология и другие. Они могут использовать SC-Web для исследования и анализа сложных знаний, а также для создания и визуализации семантических моделей и проведения анализа данных.

2 Образовательные учреждения. Университеты, школы и другие образовательные учреждения могут использовать SC-Web в качестве обучающего инструмента для студентов. Он может помочь студентам лучше понять и организовать сложные концепции и взаимосвязи в различных

предметных областях.

3 Разработчиков программного обеспечения. Разработчики могут использовать SC-Web в своих проектах для создания приложений, основанных на семантическом моделировании знаний. SC-Web может предоставить инструменты и API для интеграции с другими системами и использования семантического анализа в различных приложениях.

4 Интернет-пользователей. SC-Web может быть доступен для широкой аудитории интернет-пользователей в целях общего образования, исследования или просто удовольствия. Пользователи могут использовать SC-Web для изучения новых тем, получения информации и взаимодействия с семантическими моделями знаний.

2.3 Архитектура разрабатываемой системы

Архитектура любого проекта играет важную роль в создании масштабируемых, поддерживаемых и эффективных приложений. Хорошо спроектированная архитектура помогает в организации кода, управлении состоянием, коммуникации между компонентами и обеспечении легкой поддержки и расширяемости приложения.

SC-Web – это веб интерфейс метасистемы OSTIS IMS [5]. Ядром метасистемы является Программная модель SC-памяти (SC-machine) [6], которая одновременно может взаимодействовать как с Реализацией интерпретатора SC-моделей пользовательских интерфейсов (SC-Web), так и с любыми сторонними приложениями по соответствующим сетевым протоколам. С точки зрения общей архитектуры Реализация интерпретатора SC-моделей пользовательских интерфейсов выступает как один из множества возможных внешних компонентов, взаимодействующих с Программной моделью SC-памяти по сети (смотреть рисунок 2.1) [7].

При создании веб-интерфейсов можно использовать различные подходы и архитектурные шаблоны, такие как Flux, Redux, MobX или контекст API, чтобы достичь желаемого уровня организации и управления состоянием. Каждый из этих подходов предлагает свои принципы и инструменты для работы с данными и обновления пользовательского интерфейса.

Хорошая архитектура проекта должна учитывать принципы разделения ответственности, модульности и повторного использования кода. Это способствует легкости тестирования, обновлению и поддержке приложения на протяжении его жизненного цикла. Поэтому в процессе разработки был использован компонентный подход к созданию веб-приложений [8]. Этот подход предполагает разбиение приложения на небольшие независимые компоненты, которые могут быть повторно использованы и легко модифицированы. Каждый компонент отвечает за определенную функциональность или отображение данных.

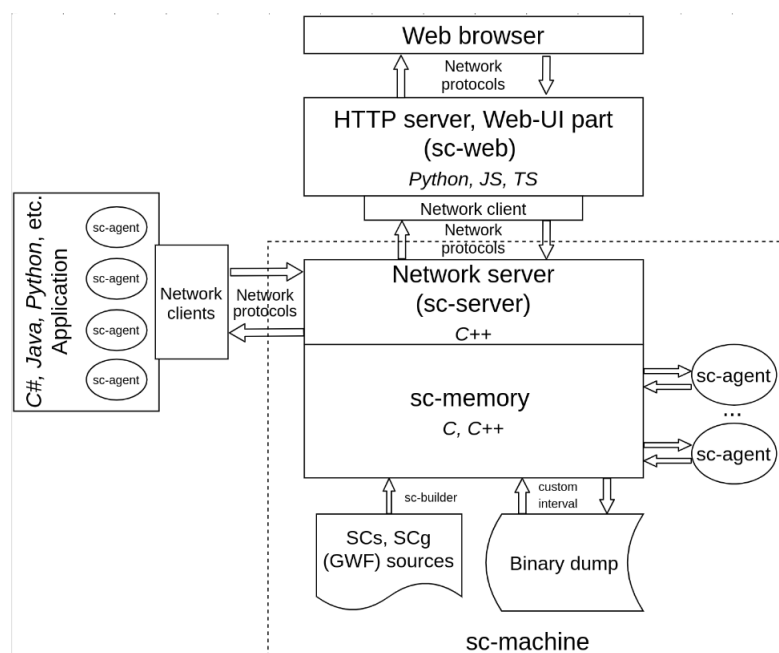


Рисунок 2.1 – Программный вариант реализации платформы интерпретации SC-моделей компьютерных систем

Были спроектированы компоненты, которые отвечают за отображение отдельных элементов пользовательского интерфейса. Основными компонентами пользовательского интерфейса являются компоненты логотипа, боковой панели и поля основного контента. Каждый компонент в свою очередь разбивается на несколько других компонентов.

Также было использовано глобальное хранилище состояния Redux, позволяющее эффективно управлять состоянием приложения и обновлять компоненты при изменении данных.

Помимо управления состоянием, архитектура проекта также включает в себя организацию компонентов, маршрутизацию, обработку событий и взаимодействие с серверной частью. Был использован паттерн контейнер-компонент, который отделяет бизнес-логику от представления и облегчает переиспользование компонентов.

В ходе проектирования были разработаны bpmn-диаграммы для разных бизнес-процессов.

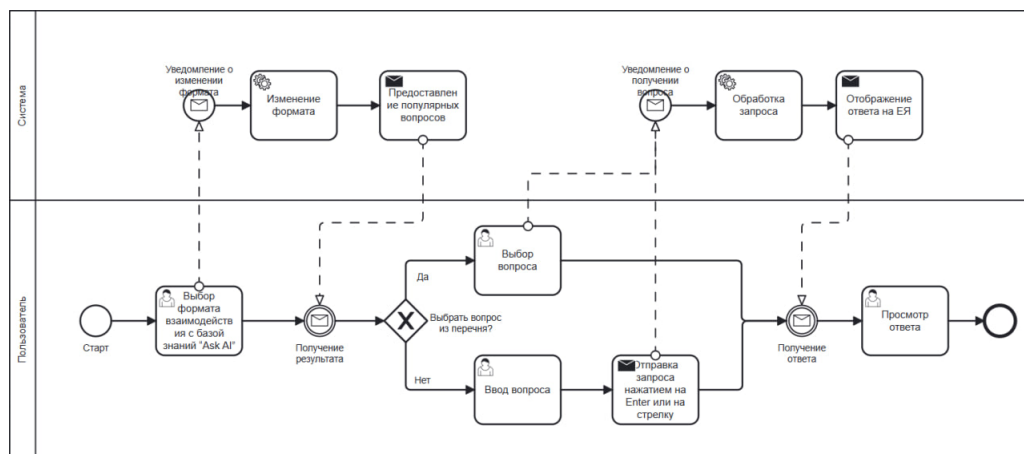


Рисунок 2.2 – Сценарий взаимодействия пользователя с режимом "Ask AI"

На рисунке 2.2 представлена последовательность шагов взаимодействия пользователя с режимом "Ask AI", который осуществляет взаимосвязь между пользователем и системой.

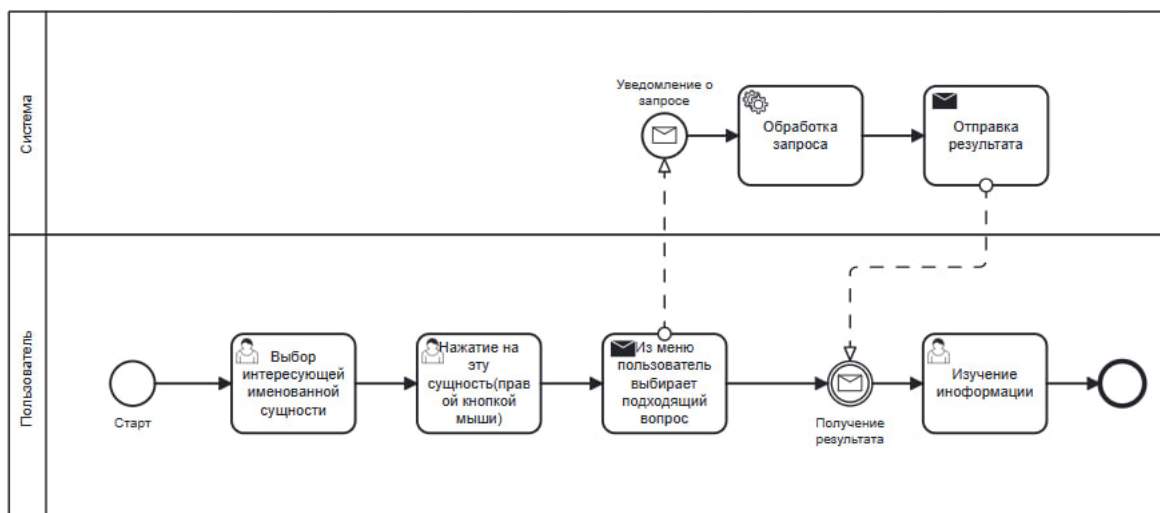


Рисунок 2.3 – Сценарий перемещения пользователя по базе знаний посредством меню

На рисунке 2.3 представлена последовательность шагов для перемещения по базе знаний посредством вызова меню аргументов (с помощью вопросов).

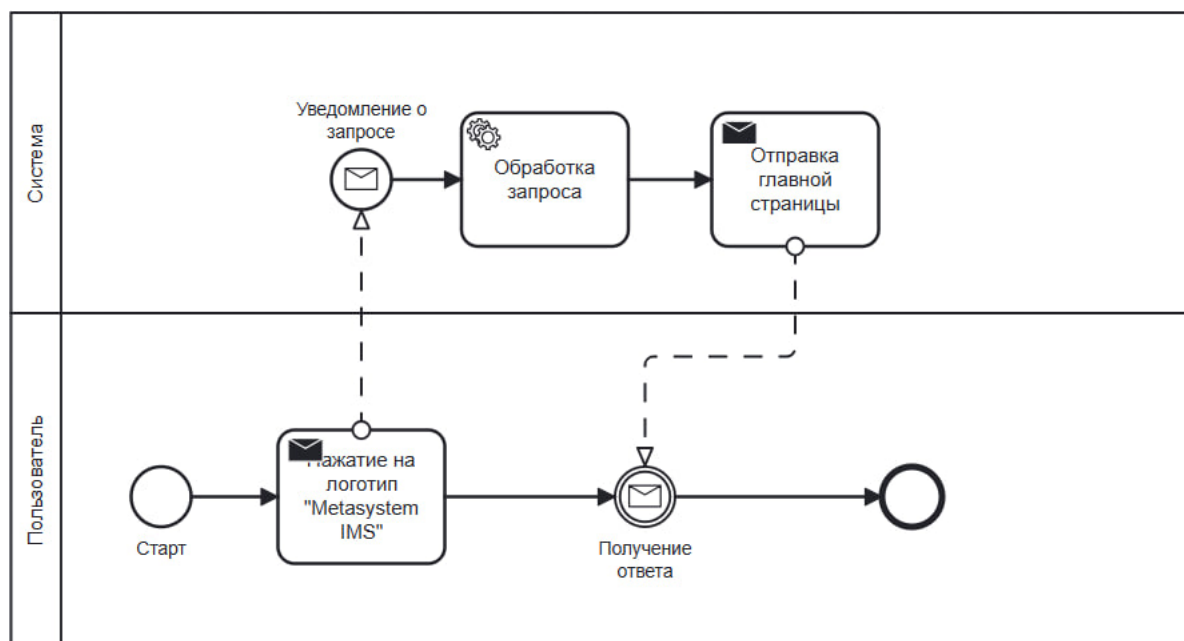


Рисунок 2.4 – Сценарий взаимодействия пользователя с логотипом

На рисунке 2.4 представлена последовательность шагов взаимодействия пользователя с логотипом, который при нажатии на него отправляет на главную страницу.

2.4 Вывод

В результате проведенного проектирования новой версии SC-Web можно сделать следующие выводы:

1 SC-Web дает возможность интуитивно понятно взаимодействовать с метасистемой OSTIS IMS, которая в свою очередь предоставляет различным группам пользователей полезные инструменты и функциональность, которые облегчают разработку, управление, анализ и использование приложений и систем. Это способствует повышению производительности, эффективности и удовлетворенности пользователей в различных ролях и областях деятельности. Для большей наглядности были предоставлены общие концепты взаимодействия с системой в BPMN диаграммах.

2 В контексте проектирования системы, был выбран компонентный подход к архитектуре. Он должен позволить доработать уже существующий функционал, а также добавить эффективную с точки зрения качества работы и последующей поддержки приложения реализацию нового режима Ask AI.

3 При проектировании новой версии SC-Web были выделены основные компоненты нового режима Ask AI. Это должно увеличить скорость на этапе разработки.

3 РАЗРАБОТКА ВЕБ-ИНТЕРФЕЙСА

3.1 Используемые средства разработки

Разрабатываемая система является веб-интерфейсом, поэтому разработка будет вестись с использованием языков программирования JavaScript и TypeScript, также языка разметки HTML и языка стилей CSS с препроцессором SASS.

JavaScript является основным языком программирования для веб-интерфейсов. Он поддерживается всеми современными браузерами и позволяет создавать динамические и интерактивные элементы на веб-страницах. JavaScript обладает широкой функциональностью, включая манипуляцию DOM, обработку событий, выполнение асинхронных запросов и многое другое.

TypeScript является суперсетом JavaScript, который добавляет статическую типизацию и другие возможности разработки, такие как интерфейсы, перечисления, декораторы и т.д. Это позволяет разработчикам обнаруживать ошибки на этапе компиляции, улучшает поддержку кода в IDE и облегчает совместную работу в больших командах. TypeScript является особенно полезным при разработке крупных проектов, где безопасность типов и поддержка инструментов для автоматического рефакторинга имеют большое значение.

HTML (HyperText Markup Language) является стандартным языком разметки для создания структуры и содержимого веб-страниц. Он определяет элементы и их взаимосвязи, такие как заголовки, параграфы, таблицы, изображения и т.д. HTML позволяет разработчикам определить семантическую структуру веб-страницы и является основой для отображения контента.

CSS (Cascading Style Sheets) используется для определения внешнего вида и стиля веб-страниц. С помощью CSS разработчики могут задавать цвета, шрифты, размеры, расположение элементов и другие аспекты визуального представления страницы. CSS позволяет создавать красивый и согласованный дизайн, а также обеспечивает адаптивность и респонсивность интерфейса для различных устройств и экранов.

SASS (Syntactically Awesome Style Sheets) является метаязыком, расширяющим возможности языка CSS. Он предоставляет разработчикам мощные инструменты и функции для более эффективного и организованного написания стилей веб-приложений.

При анализе вышеперечисленных пунктов в качестве основного фреймворка был выбран React [9] в связке с Redux [10]. Так как React и Redux являются популярными инструментами с большим сообществом разработчиков. Это означает, что есть множество готовых решений, библиотек, утилит

и документации, которые помогут в разработке и решении типичных задач. Также React известен своим компонентным подходом, что позволяет создавать масштабируемые и легкоподдерживаемые веб приложения

3.2 Реализация функционала

Для реализации были разработаны компоненты AskElement и AskAnswer.

```
1 import axios from 'axios';
2
3 import { useLocation } from 'react-router-dom';
4 import { useEffect, useReducer, useState } from 'react';
5
6 import { Spinner, useLanguage } from 'ostis-ui-lib';
7 import { SPINER_COLOR } from '@constants';
8 import { AskElement } from './AskElement';
9
10 import styles from './AskAnswer.module.scss';
11 import { AskInput } from '@components/AskInput';
12
13 import { getHintButtonHandler } from 'src/constants/hintButtons';
14
15 interface NavigateState {
16   query?: string;
17   isHintButton: boolean;
18 }
19
20 interface ElementPayload {
21   query: string;
22   answer: string;
23 }
24
25 interface State {
26   history: ElementPayload[];
27 }
28
29 type Action = { type: 'ADD'; payload: ElementPayload } | { type: 'RESET' };
30
31 const historyReducer = (state: State, action: Action): State => {
32   switch (action.type) {
33     case 'ADD':
34       return {
35         ...state,
36         history: [...state.history, action.payload],
37       };
38     case 'RESET':
39       return {
40         ...state,
41         history: [],
42       };
43     default:
44       throw new Error('Unknown action type');
45   }
46 };
47
48 export const AskAnswer = () => {
49   const state = useLocation().state as NavigateState;
50
51   const lang = useLanguage();
52
53   const [historyState, dispatch] = useReducer(historyReducer, { history: [] });
54
55   const [isLoading, setIsLoading] = useState(false);
56
57   const [query, setQuery] = useState<string>();
58
59   const onInputChange = (value: string) => setQuery(value);
60   const onInputSubmit = async () => await fetchAnswerByQuery(query);
61
62   const isDescribeQuery = (query: string): boolean => {
63     const starts =
64       lang === 'ru'
65       ? ['Что это', 'Что это такое', 'Расскажи про']
66       : ['What is', 'Tell me about', 'Describe'];
67     return starts.some((start) => query.startsWith(start));
68   };
69 }
```

Рисунок 3.1 – Реализация компонента AskAnswer

Компонент AskAnswer является частью функционала, предоставляющего возможность пользователям получать ответы от метасистемы OSTIS IMS. Реализация данного компонента изображена на рисунках 3.1 и 3.2.

Компонент AskAnswer использует хук useState для эффективного управления состоянием. Хук useState позволяет сохранять и обновлять значения переменных внутри компонента. В данном случае, используется состояние query, которое представляет текущий вопрос, заданный пользователем.

Кроме того, на странице присутствует компонент AskInput, который предоставляет пользователю возможность вводить свои собственные вопросы. При изменении значения в поле ввода, вызывается функция onChange, которая обновляет состояние query. При отправке формы вызывается функция onSubmit, которая выполняет запрос на получение ответа на вопрос и обновляет состояние истории.

```
70 const fetchAnswerByQuery = async (query: string | undefined) => {
71   console.log(query);
72   if (!query) return;
73
74   setIsLoading(true);
75
76   let answer: string | null = null;
77
78   if (state.isHintButton) {
79     const handler = getHintButtonHandler(lang, state.query!);
80     console.log(handler);
81     answer = await handler();
82   } else if (isDescribeQuery(state.query)) {
83     answer = await getDescriptionOfElement(state.query);
84   }
85
86   if (answer) {
87     dispatch({ type: 'ADD', payload: { query, answer } });
88   } else {
89     dispatch({ type: 'ADD', payload: { query, answer: 'Failed to fetch data' } });
90   }
91
92   setIsLoading(false);
93 };
94
95 useEffect(() => {
96   if (state.query) {
97     fetchAnswerByQuery(state.query);
98   }
99 }, [state]);
100
101 if (!state) return null;
102
103 if (isLoading) return <Spinner className={styles.spinner} appearance={SPINER_COLOR} />;
104
105 return (
106   <div className={styles.pageWrapper}>
107     <div className={styles.history}>
108       {historyState.history.map((entry) => (
109         <AskElement key={entry.query} {...entry} />
110       ))}
111     </div>
112     <AskInput
113       className={styles.inputQueryBar}
114       onChange={onChange}
115       onSubmit={onSubmit}
116     />
117   </div>
118 );
119 };
120
```

Рисунок 3.2 – Реализация компонента AskAnswer

Функция `isDescribeQuery` используется для определения, является ли запрос описанием. В зависимости от типа запроса, выполняется соответствующее действие для получения ответа.

Компонент также использует хук `useEffect` для выполнения запроса при изменении состояния. Если состояние `state` содержит запрос, то вызывается функция `fetchAnswerByQuery` для получения ответа на запрос.

Если компонент находится в состоянии загрузки (`isLoading` равно `true`), отображается компонент `Spinner`. В противном случае, отображается история запросов и компонент `AskInput` для ввода нового запроса.

На рисунке 3.3 изображена реализация компонента `AskElement`.

```
1 import { FC } from 'react';
2 import styles from './AskElement.module.scss';
3
4 import AskAIMessageIcon from '@assets/images/AskAIMessageIcon.svg';
5
6 interface IProps {
7   query: string;
8   answer: string;
9 }
10
11 export const AskElement: FC<IProps> = ({ query, answer }) => {
12   return (
13     <div className={styles.elementWrapper}>
14       <div className={styles.elementQuery}>{query}</div>
15       <div className={styles.elementAnswer}>
16         <div>{<AskAIMessageIcon />}</div>
17         <div>{JSON.stringify(answer)}</div>
18       </div>
19     </div>
20   );
21 };
22
```

Рисунок 3.3 – Реализация компонента `AskElement`

Компонент `AskElement` представляет отдельный элемент истории запросов. Он принимает два свойства - `query` и `answer`, которые отображают текущий вопрос и ответ соответственно.

Внутри компонента имеется обертка `div` с классом `elementWrapper`, которая содержит два дочерних элемента. Первый дочерний элемент с классом `elementQuery` отображает текст текущего вопроса (`query`).

Второй дочерний элемент с классом `elementAnswer` представляет ответ на вопрос. Он состоит из двух частей. Первая часть содержит компонент `AskAIMessageIcon`, который отображает иконку сообщения от искусственного интеллекта. Вторая часть отображает текстовое представление ответа (`answer`), преобразованное в строку с помощью функции `JSON.stringify`.

Для описания компонентов интерфейса в БЗ был создан раздел "Предметная область интерфейса OSTIS IMS". Этот раздел включает в себя

различные сущности, такие как кнопка, поле для ввода сообщений, поле для вывода сообщений и другие компоненты, которые могут присутствовать в интерфейсах.

С использованием этого раздела был описан сам интерфейс Metasystem IMS. Для каждого компонента интерфейса были указаны его свойства, функциональные возможности и внешний вид. В описании учтены такие детали, как стили текста, размеры и цвета компонентов (смотреть рисунки 3.4 и 3.5).

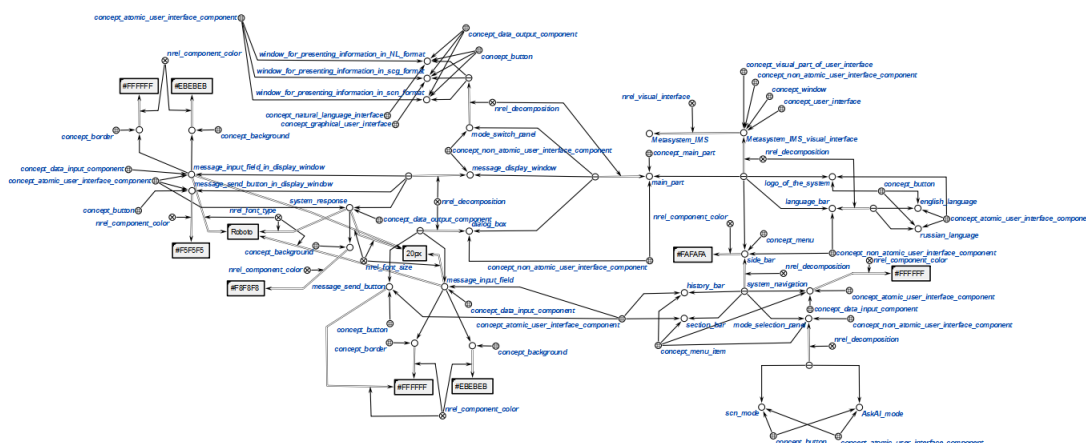


Рисунок 3.4 – Описание интерфейса Metasystem IMS

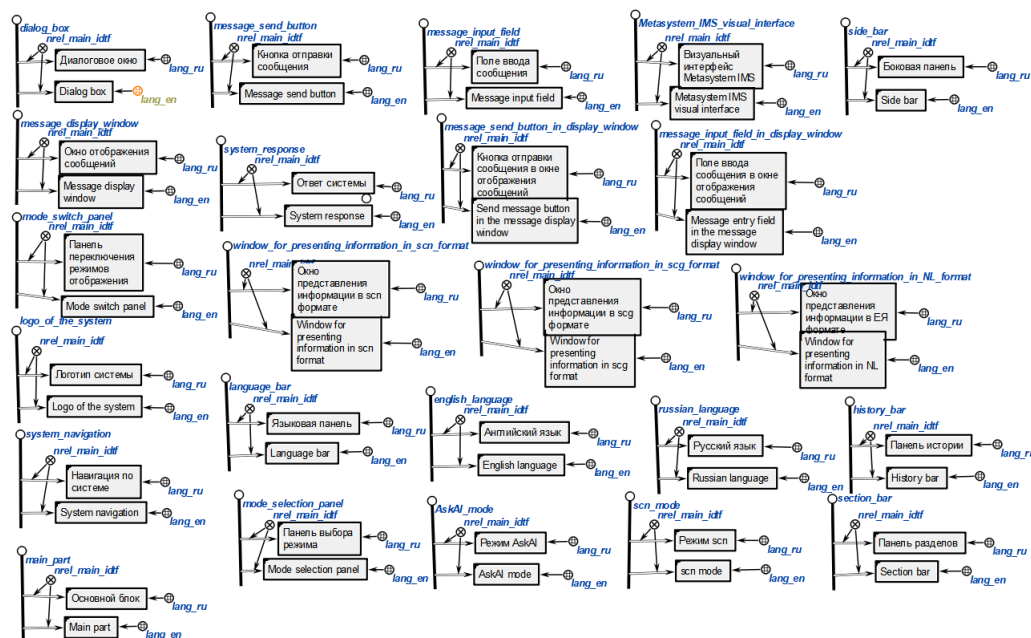


Рисунок 3.5 – Компоненты интерфейса

Таким образом, БЗ содержит информацию о различных интерфейсных компонентах, а их использование в описании интерфейса Metasystem IMS позволяет предоставить полное представление о том, как пользователи

могут взаимодействовать с интерфейсом и какие компоненты доступны для использования. Это облегчает понимание и разработку интерфейса Metasystem IMS.

3.3 Демонстрация реализованного функционала

В ходе работы был реализован логотип компании и компонент диалога. Компонент диалога был реализован для взаимодействия системы с пользователем. Он представляет собой интерфейс, через который пользователь может задавать вопросы, получать ответы.

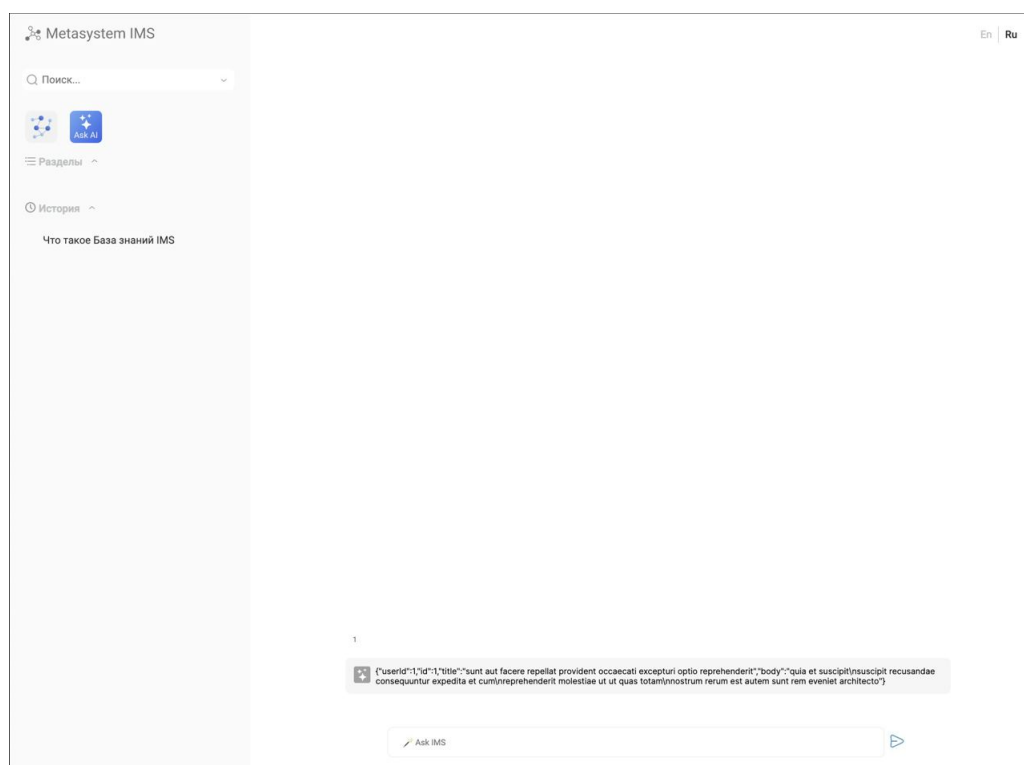


Рисунок 3.6 – Демонстрация основного функционала

Пользователь может вводить свои запросы в компонент диалога, используя текстовое поле. Система, основанная на искусственном интеллекте, анализирует введенные пользователем запросы и предоставляет ответы на основе имеющейся базы знаний. Ответы отображаются в виде текста (смотреть рисунок 3.6).

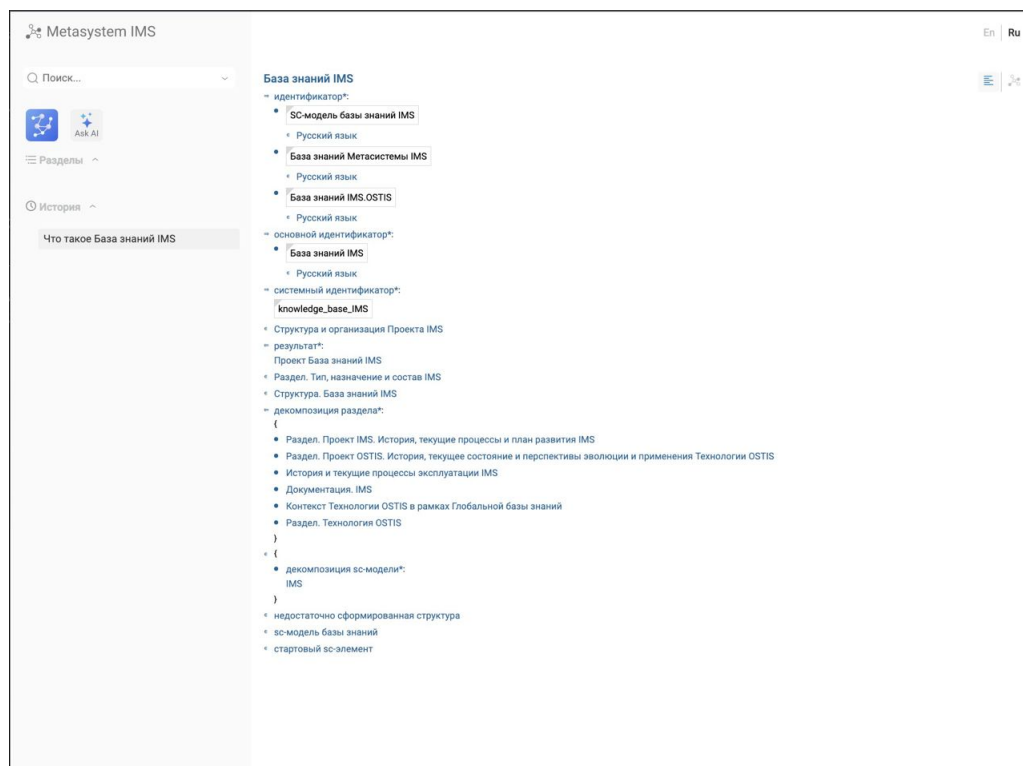


Рисунок 3.7 – Открытие главного экрана системы после нажатия на логотип

Логотип представлен графическим изображением, которое может быть использовано для идентификации компании и создания ее узнаваемого образа. При его нажатии открывается главный экран системы (смотреть рисунок 3.7). Это позволяет обеспечить пользователю легкий доступ к основным функциям и навигационным элементам платформы.

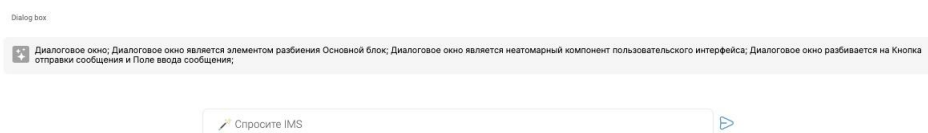


Рисунок 3.8 – Получение информации о компоненте интерфейса

После добавления описания компонентов интерфейса в БЗ пользователь может ввести интересующий его компонент чтобы увидеть его описание. Это позволяет пользователям получать информацию о компонентах интерфейса в удобной и понятной форме. Пример получения информации о диалоговом окне показан на рисунке 3.8.

3.4 Вывод

По результатам разработки новой версии SC-Web можно сделать следующие выводы:

- в качестве основных инструментов разработки были выбраны языки программирования JavaScript и TypeScript, также языки разметки HTML и CSS (с препроцессором SASS);
- в качестве основных фреймворков для разработки были выбраны React и Redux;
- при разработке новой версии SC-Web был реализован логотип компании и компонент диалога для взаимодействия пользователя с системой;
- был продемонстрирован основной функционал разрабатываемой системы, а именно логотип, который служит для идентификации компании и открытия главного экрана системы при его нажатии, компонент диалога, который обеспечивает взаимодействие системы с пользователем, а также было добавлено описание компонентов интерфейса в БЗ

ЗАКЛЮЧЕНИЕ

Целью курсового проекта была успешная реализация интерпретатора SC-моделей пользовательских интерфейсов, с акцентом на улучшение пользовательского опыта, гибкой архитектуре и расширенных функциональных возможностях.

В начале разработки был проведен анализ требований к системе, определены необходимые возможности, которые должна предоставлять система пользователю. Также был проведен анализ аналогов, выявлены преимущества и недостатки разрабатываемой системы по сравнению с аналогами. Это структурированное и связанное представление информации сыграло ключевую роль в более эффективной разработке новой версии SC-Web.

Затем были выделены основные подзадачи, необходимые для достижения желаемого результата, и определены потенциальные пользователи разрабатываемой системы. Это позволило создать подробные описания и учесть потребности различных категорий пользователей.

В результате была разработана и описана архитектура системы, основанная на компонентном подходе с использованием React и Redux для управления состоянием. Эта спроектированная архитектура обеспечивает гибкую и удобную структуру, которая легко может быть расширена и дополнена в будущем.

Выбор языка программирования TypeScript для реализации проекта интерпретатора SC-моделей пользовательских интерфейсов был обоснованным и имел ряд преимуществ. TypeScript является надмножеством языка JavaScript, добавляющим статическую типизацию и другие возможности, что способствует повышению надежности и поддерживаемости кода. Статическая типизация позволяет выявлять ошибки на этапе разработки и обеспечивает более ясное понимание структуры и взаимодействия компонентов системы.

HTML и CSS были выбраны для разметки и стилей пользовательского интерфейса. HTML является стандартным языком разметки для создания структуры веб-страниц, а CSS позволяет определить внешний вид и стиль элементов на странице. Использование HTML и CSS в проекте обеспечивает гибкость и совместимость с различными браузерами и устройствами.

Для улучшения процесса разработки и поддержки стилей был применен препроцессор SASS (Syntactically Awesome Style Sheets). SASS предоставляет дополнительные возможности, такие как переменные, миксины и вложенность, что упрощает написание и организацию стилей. Он также позволяет использовать функции и операторы, что облегчает создание динамических и адаптивных стилей.

Ожидается, что новая версия SC-Web станет полезным инструментом

для исследования и анализа знаний, а также найдет широкое применение среди образовательных учреждений, разработчиков программного обеспечения и интернет-пользователей.

Кроме того, после завершения разработки и успешной реализации интерпретатора SC-моделей пользовательских интерфейсов, была проведена оценка и анализ полученных результатов. Это позволило выявить достигнутые цели проекта, а также выделить области для дальнейшего улучшения и развития системы.

Курсовой проект по разработке интерпретатора SC-моделей пользовательских интерфейсов привел к созданию улучшенной версии SC-Web, которая предлагает новые возможности и может служить полезным инструментом в различных областях.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Садовский, М. Е. Пользовательский интерфейс экосистемы OSTIS / М. Е. Садовский. — Минск : БГУИР, 2023. — 153 – 158 с.
- [2] Open Semantic Technology for Intelligent Systems [электронный ресурс]. — Режим доступа: <http://conf.ostis.net/>. — Дата доступа: 01.11.2023.
- [3] Chatsonic.com [электронный ресурс]. — Режим доступа: <https://chatsonic.com/>. — Дата доступа: 16.10.2023.
- [4] Rytr.com [электронный ресурс]. — Режим доступа: <https://rytr.com/>. — Дата доступа: 14.10.2023.
- [5] OSTIS IMS. — Режим доступа: <https://ims.ostis.net/>. — Дата доступа: 14.10.2023.
- [6] Реализация платформы для web-ориентированных систем, управляемых знаниями [электронный ресурс]. — Режим доступа: https://libeldoc.bsuir.by/bitstream/123456789/3938/1/Koronchik_Realizatsiya.PDF. — Дата доступа: 20.10.2023.
- [7] Документация программного варианта реализации платформы интерпретации sc-моделей компьютерных систем [электронный ресурс]. — Режим доступа: <https://github.com/ostis-ai/ostis-web-platform/blob/develop/docs/main.pdf>. — Дата доступа: 20.10.2023.
- [8] What is Component-Based Architecture? [электронный ресурс]. — Режим доступа: <https://www.mendix.com/blog/what-is-component-based-architecture/>. — Дата доступа: 01.11.2023.
- [9] React Documentation. — Режим доступа: <https://legacy.reactjs.org/docs/getting-started.html>. — Дата доступа: 17.10.2023.
- [10] Redux Documentation. — Режим доступа: <https://redux.js.org/>. — Дата доступа: 17.10.2023.